BUSINESS REQUIREMENTS DOCUMENT

CloudArena

A cloud-powered 2D arcade fighting game - project plan

Document control	
Version	1.0
Date	16 October 2026
Development team	Purab Gaonkar, Saniya Bhartu
Game engine	Godot 4 (GDScript)
Cloud platform	Google Cloud Platform (Cloud Run, Memorystore Redis, Terraform)

1. Project overview

CloudArena is a 1v1 arcade fighting game in the spirit of Street Fighter and Tekken. Two fighters battle across best-of-N rounds on atmospheric, layered stages, using combos, heavy attacks, blocks, throws, dashes and a cinematic super move. The game will be playable offline against the computer or a friend on the same PC, and online against players anywhere through a cloud-hosted, server-authoritative backend.

> **Vision:** a polished, replayable fighting game that works fully offline and scales to online play on cloud infrastructure that is cheap when idle and elastic under load.

2. Business objectives

ID	Objective	How success will be measured
BO-1	Deliver a complete, polished single-machine game	New player reaches a finished match within 60 seconds of launch; automated full-match soak tests run without errors.
BO-2	Give combat real depth	At least eight distinct combat actions, each with counter-play.
BO-3	Work on any input device	Complete play on keyboard and gamepad with every control rebindable.
BO-4	Professional audiovisual presentation	Animated menus, layered stages, consistent HUD, music and sound on every screen.
BO-5	Online multiplayer that scales	Matchmaking in constant time; rooms spread across instances; stable 60 Hz server simulation.
BO-6	Low operating cost	Scale-to-zero compute and one small Redis instance at launch.

3. Scope

3.1 In scope

- Desktop game client built with Godot 4 (Windows; Linux build for the dedicated server).
- Game modes: VS CPU, Local 2-Player, Training, Online Match.
- Three fighters, four stages, configurable round rules.
- Gamepad support and fully rebindable controls.
- Music, sound effects and volume settings from licence-compatible assets.
- Cloud backend: WebSocket game server, matchmaking and statistics API, Redis, NGINX, Terraform deployment.
- Automated testing, build/export pipeline and release packaging.

3.2 Out of scope

- Mobile and console ports.

- Player accounts, payments, cosmetic store, ranked ladders.
- Replays, spectator mode and tournament tools.
- Rollback netcode (the first online release is server-authoritative).

4. Planned features

Priority: **MUST** required for release, **SHOULD** important, **COULD** if time allows.

4.1 Game modes and flow

ID	Feature	Priority
F-01	Animated loading screen with real asset-loading progress, then fade to the main menu	**SHOULD**
F-02	Main menu with animated arena backdrop: VS CPU, Local 2P, Training, Online, Controls, Options, Credits, Quit	**MUST**
F-03	Character select with stage choice and colour palettes	**MUST**
F-04	Match rules: 1, 2 or 3 rounds to win; 99/60/30 s or unlimited timer; draw handling	**MUST**
F-05	Round presentation: VS intro, ROUND / FINAL ROUND, FIGHT, K.O., TIME UP, PERFECT, win banners	**SHOULD**
F-06	Victory screen with match statistics (rounds, damage, best combo, supers, time); rematch and navigation options	**SHOULD**
F-07	Pause menu with move list	**SHOULD**
F-08	Training mode with configurable dummy (stand / block / fight back), infinite meter and live damage readout	**SHOULD**
F-09	CPU opponent with easy / normal / hard behaviour	**MUST**

4.2 Combat

ID	Feature	Priority
F-10	Three-hit light combo with launching finisher	**MUST**
F-11	Heavy attack with charge-up and large knockback	**MUST**
F-12	Block with chip damage; attacks from behind unblockable	**MUST**
F-13	Parry: precisely timed block negates damage and staggers the attacker	**SHOULD**
F-14	Throw (attack + block) that beats defensive play	**SHOULD**
F-15	Dash and invulnerable backdash (double-tap)	**SHOULD**
F-16	Counter hits (bonus damage during attack start-up), combo damage scaling, wake-up invulnerability	**SHOULD**
F-17	Super meter and one cinematic super per fighter. Telegraphed start-up; the freeze-frame cinematic plays only when the first hit connects, so the defender can dodge	**MUST**

F-18	Hit feedback: hit-stop, screen shake, camera punch, particles, floating damage numbers, combo counter, slow-motion K.O.	**MUST**
F-19	Fighters push apart instead of colliding so nobody can stand on another	**MUST**

4.3 Fighters, stages and visuals

ID	Feature	Priority
F-20	Three fighters with distinct stats and supers: Brawler (balanced), Samurai (long reach), Shinobi (fast)	**MUST**
F-21	Four stages with parallax depth layers, light rays, ambient particles, blurred foreground planes and vignette	**MUST**
F-22	Arcade HUD: slanted health bars with damage trail, portraits, super meter, round pips, timer, announcer banners	**MUST**
F-23	Dynamic camera with zoom, parallax and cinematic moments	**SHOULD**
F-24	Extra move and effect sprites produced by a reproducible generator script, replaceable by hand-drawn art	**SHOULD**

4.4 Input, audio and settings

ID	Feature	Priority
F-30	Gamepad support for both players, including menu navigation	**MUST**
F-31	Controls menu to rebind every action for keyboard and gamepad; saved between sessions	**MUST**
F-32	Music per screen, pooled sound effects, jingles, volume sliders	**MUST**
F-33	Options: CPU difficulty, rounds, round time, screen shake, fullscreen, foreground blur	**SHOULD**
F-34	First-match controls tip and in-game credits screen	**COULD**

4.5 Online multiplayer and cloud

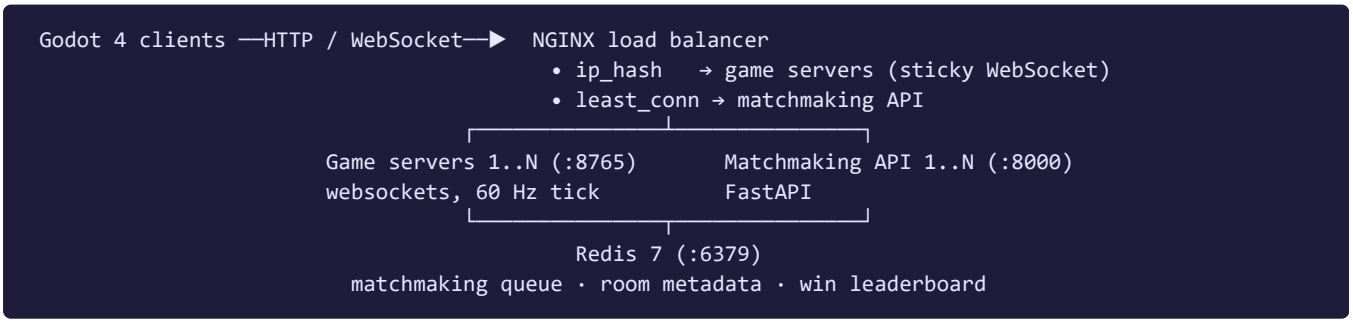
ID	Feature	Priority
F-40	Matchmaking API: join returns a room and WebSocket address; open rooms handed out in constant time via a Redis list; rooms expire after 10 minutes	**MUST**
F-41	Game server: 60 Hz authoritative simulation per room, state broadcast, separate HTTP health endpoint	**MUST**
F-42	Network protocol: client sends inputs only (move, jump, attack, heavy, block, super, dash, throw); server validates the sender and returns state	**MUST**
F-43	Client online flow: matchmaking screen, connect, wait-for-opponent, in-match HUD, disconnect handling	**MUST**
F-44	Replicate the full combat feature set online: chosen fighter and stage, meter, supers, parry, throw, dash, round and match flow driven by the server	**MUST**

F-45	Match results feed a global win leaderboard (`/api/stats/result` , `/api/stats/leaderboard` , `/api/stats/player/{name}`) and an in-game leaderboard screen	**SHOULD**
F-46	Keepalive pings every 20 seconds to survive the Cloud Run idle timeout; reconnect on short drops	**MUST**
F-47	NGINX load balancing: sticky WebSocket routing (`ip_hash`), least-connection API routing, health endpoint	**MUST**
F-48	One-command infrastructure with Terraform: Cloud Run services, Memorystore Redis, VPC connector, Artifact Registry; Docker Compose for local development	**MUST**
F-49	Dedicated headless server export for Linux	**MUST**
F-50	Structured JSON logging, health checks, dashboards and alerts in Cloud Logging / Monitoring	**SHOULD**

5. Technology stack

Layer	Technology	Purpose
Game engine	Godot 4.x, GDScript	Rendering, physics, input, UI, audio, scene management, networking client.
Client networking	WebSocketMultiplayerPeer, RPC, MultiplayerSynchronizer	Server-authoritative gameplay with state replication.
Asset tooling	Python 3, Pillow, NumPy	Generates extra move and effect sprites reproducibly.
Game server	Python 3.11, `websockets`, `redis.asyncio`	Real-time room simulation at 60 Hz; HTTP health probe on a separate port.
Matchmaking / stats API	FastAPI, Uvicorn, Pydantic	Room assignment, match state, results, leaderboard.
Shared state	Redis 7 (Memorystore in the cloud)	Matchmaking queue, room keys with TTL, sorted-set leaderboard.
Load balancer	NGINX 1.27	Sticky WebSockets and balanced REST traffic.
Containers	Docker, Docker Compose	Reproducible local backend stack.
Cloud hosting	GCP: Cloud Run v2, Memorystore, Serverless VPC Access, Artifact Registry	Autoscaling, scale-to-zero hosting.
Infrastructure as code	Terraform 1.5+, gcloud CLI	Repeatable provisioning and teardown.
Quality	Godot headless test scripts, pytest, load-testing tool (k6 or Locust), GitHub Actions	Automated gameplay soak tests, API tests, WebSocket load tests, CI builds.
Source control	Git, pull-request workflow	Collaboration between the two developers.

Architecture



Design principles: the server owns the simulation and clients send inputs only; services are stateless and scale by adding instances; all shared state lives in Redis; infrastructure is fully described as code.

6. Non-functional requirements

Category	Requirement
Performance	60 FPS or better at 1280x720 on integrated graphics; server tick 60 Hz.
Responsiveness	Local input-to-action latency within one physics frame; online round-trip target under 100 ms in region.
Reliability	Zero script errors across automated full-match soak and random-input tests; graceful handling of missing optional assets and corrupt settings.
Scalability	Stateless containers behind a balancer; add instances to add capacity.
Security	Server validates every input and its sender; no secrets in the client; least-privilege cloud service accounts.
Usability and accessibility	Readable HUD, full gamepad parity, rebindable controls, screen-shake toggle, adjustable audio.
Maintainability	Data-driven fighters and stages, shared UI module, documented repository guide, reproducible tooling.
Observability	JSON logs, health endpoints on all services, dashboards and alerts.
Legal	Only assets with compatible licences; credits screen and credits file kept current.
Cost	Scale-to-zero compute, smallest viable Redis tier, budget alerts, one-command teardown.

7. Delivery plan

Phase	Duration	Deliverables
1. Foundations	Weeks 1-2	Repository workflow, coding standards, client/server message contract, local Docker stack, base combat, HUD and round flow.
2. Local game	Weeks 3-6	All three fighters, combat set (F-10 to F-19), CPU, training, stages, menus, audio, settings, gamepad and rebinding.
3. Backend services	Weeks 3-6 (parallel)	Matchmaking API, game server rooms, Redis model, NGINX, Terraform and Cloud Run deployment, CI.
4. Online integration	Weeks 7-9	Client online flow, full feature replication, server-driven rounds, reconnect handling, leaderboard reporting and screen.
5. Hardening	Weeks 10-11	Load and soak tests, latency tuning, balance pass, monitoring and alerts, bug fixing.
6. Release	Week 12	Windows and Linux builds, deployment to production project, release notes, documentation, post-launch checklist.

8. Assumptions, constraints and dependencies

- **Assumptions:** players use Windows 10/11 with a Direct3D 12 or Vulkan GPU; online play targets a single GCP region at first; two developers are available throughout.
- **Constraints:** Cloud Run closes idle WebSockets after 5 minutes; connections must reach the same instance as their room; extra move art is generated from existing sprites rather than hand-drawn; fixed team size.
- **Dependencies:** Godot 4, a GCP project with billing, Docker, Terraform, and the free/CC0 art and audio packs listed in section 10.

9. Risks and mitigations

#	Risk	Impact	Mitigation
1	Latency makes online combat feel sluggish	High	Server-authoritative first, forgiving timing windows, region choice; evaluate prediction or rollback later.
2	Client and server drift apart on combat rules	High	Single shared message contract, resolve all hits server-side, integration tests every week.
3	WebSocket idle timeouts or affinity failures on Cloud Run	Medium	20 s keepalive, sticky routing, health checks and room expiry.
4	Cloud cost grows with players	Medium	Scale-to-zero, small Redis tier, budget alerts, Terraform teardown.
5	Balance issues or exploits	Medium	Data-driven stats, combo scaling, wake-up invulnerability, training mode and automated soak tests.

6	Asset licence problems	Medium	Use CC0 or clearly licensed packs only; maintain credits.
7	Generated sprites look weaker than hand-drawn art	Medium	Stable file names so replacement art drops in without code changes.
8	Schedule slips with a small team	Medium	Parallel client and backend tracks, MUST features first, COULD features cut first.

10. Third-party assets

Asset	Source	Licence
Fighter sprites	Craftpix.net free sprite packs	Craftpix free licence
Music	OpenGameArt.org (wipics, Kistol, pmiller, Xythe / mutkanto)	CC0
Sound effects and jingles	Kenney.nl (Impact, Interface, Music Jingles)	CC0
Engine	Godot Engine	MIT

11. Acceptance criteria

1. From a cold start the game reaches the main menu and a VS CPU match begins within 60 seconds with no errors.
2. Every combat action works for both players on keyboard and gamepad; all bindings can be changed and persist.
3. A super can be dodged; its cinematic appears only when the first hit connects; a hit during start-up cancels it.
4. Two players on different machines find each other through matchmaking and finish a full online match with the same rules as local play.
5. Match results appear on the global leaderboard.
6. The backend deploys from scratch with Terraform and scales out under a load test without dropped matches.
7. Automated test suites (gameplay soak, API, WebSocket load) pass on every release build.

Appendix - Planned default controls

Action	Player 1	Player 2	Gamepad
Move / Jump	A D / W	Left Right / Up	D-pad or stick / A
Attack (tap x3 = combo)	J	/ or Numpad 0	X
Heavy attack	U	, or Numpad 1	Y
Block / Parry	S or K	Down or .	LB / LT
Throw	Attack + Block	Attack + Block	X + LB

Super finisher	I	M or Numpad 2	B / RB / RT
Dash / Backdash	Double-tap A / D	Double-tap Left / Right	Double-tap D-pad
Pause	Esc		Start

End of document - CloudArena BRD v1.0